

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:10

Total Marks:50

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink that reads "K. Vijay Bhaskar Reddy". The signature is written in a cursive style with a horizontal line underlining the last name "Reddy".

Question 1

Application Scenario

A smart city surveillance system processes continuous video feeds from multiple cameras. During peak hours, large volumes of data are transferred between I/O devices, memory, and CPU. As a result, the system experiences lag and delayed threat detection.

Identify

The major components involved are:

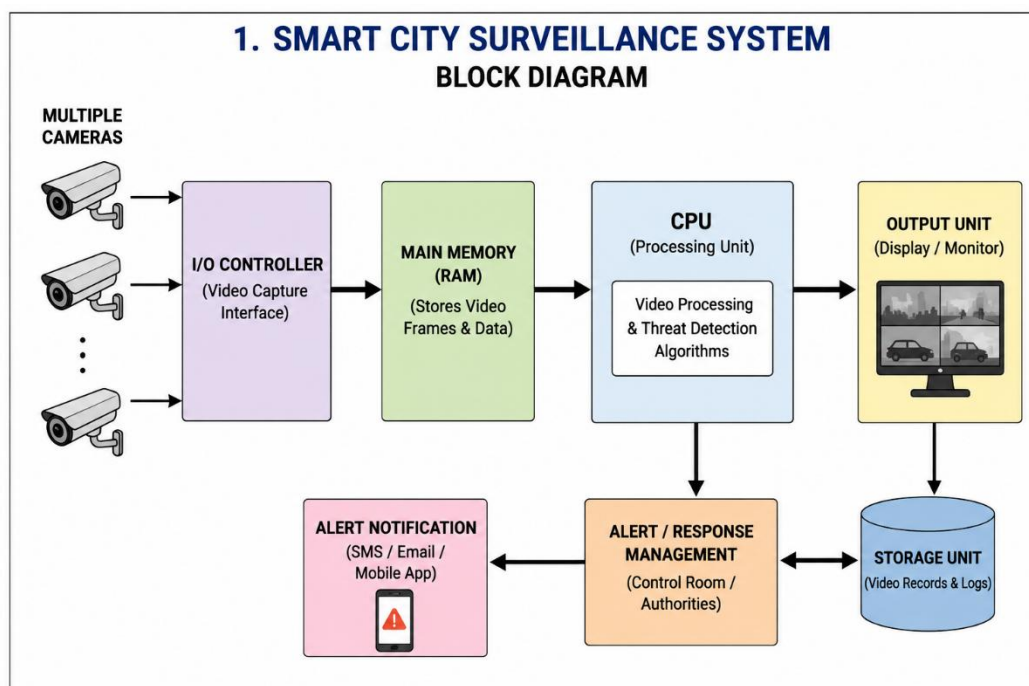
- **CPU** – Processes video data and executes threat detection algorithms.
- **Memory (RAM)** – Temporarily stores video frames and intermediate processing data.
- **I/O Devices** – Cameras continuously send video streams to the system.
- **System Bus** – Transfers data between CPU, memory, and I/O devices.

Analyze

When many cameras send video simultaneously, a huge amount of data must move through the system.

- The **CPU waits** until the required data reaches memory.
- **Memory becomes busy** storing and retrieving video frames.
- **I/O devices continuously generate new data**, increasing traffic.
- During peak hours, these operations compete for bus access, creating a **bottleneck**.

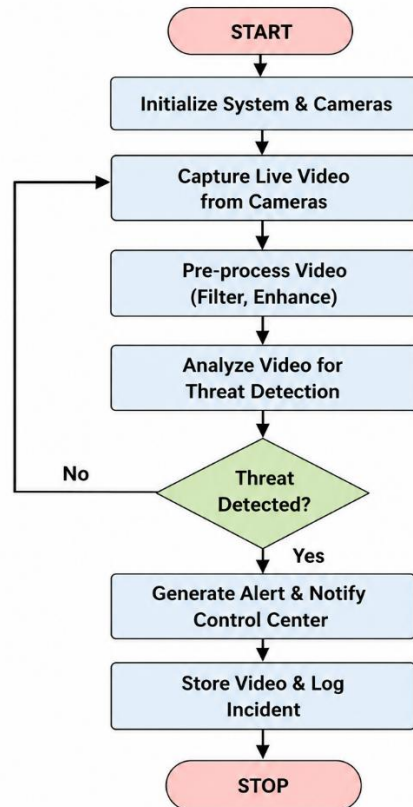
This increases response time and delays threat detection.



Comparison Table

Feature	Single Bus	Multi Bus
Data Transfer	One transfer at a time	Multiple transfers simultaneously
CPU Waiting Time	High	Low
Performance	Moderate	High
Cost	Low	High
Suitable For	Small systems	Real-time systems

FLOWCHART



Recommendation

- Use **multi-bus architecture**.
- Increase CPU cache memory.
- Use **DMA (Direct Memory Access)**.
- Implement **instruction pipelining**.
- Use faster RAM.

Justification

A multi-bus architecture allows the CPU, memory, and I/O devices to communicate simultaneously. This reduces waiting time, increases throughput, and enables faster threat detection.

Question 2

Application Scenario

A real-time stock trading application executes millions of instructions every second. During heavy trading, users experience delays because the processor has a high CPI (Cycles Per Instruction).

Identify

Execution cycle consists of:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

Analyze

Every instruction passes through these stages.

Frequent:

- Memory access
- Branch instructions
- Cache misses

increase the CPU waiting time, resulting in higher CPI.

Higher CPI means more execution time.

Calculation

CPU Performance Formula

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Example

Instruction Count = 5 Million

CPI = 2

Clock Cycle = 1 ns

CPU Time

$$= 5,000,000 \times 2 \times 1 \text{ ns}$$

= **10 ms**

If CPI reduces to **1.2**

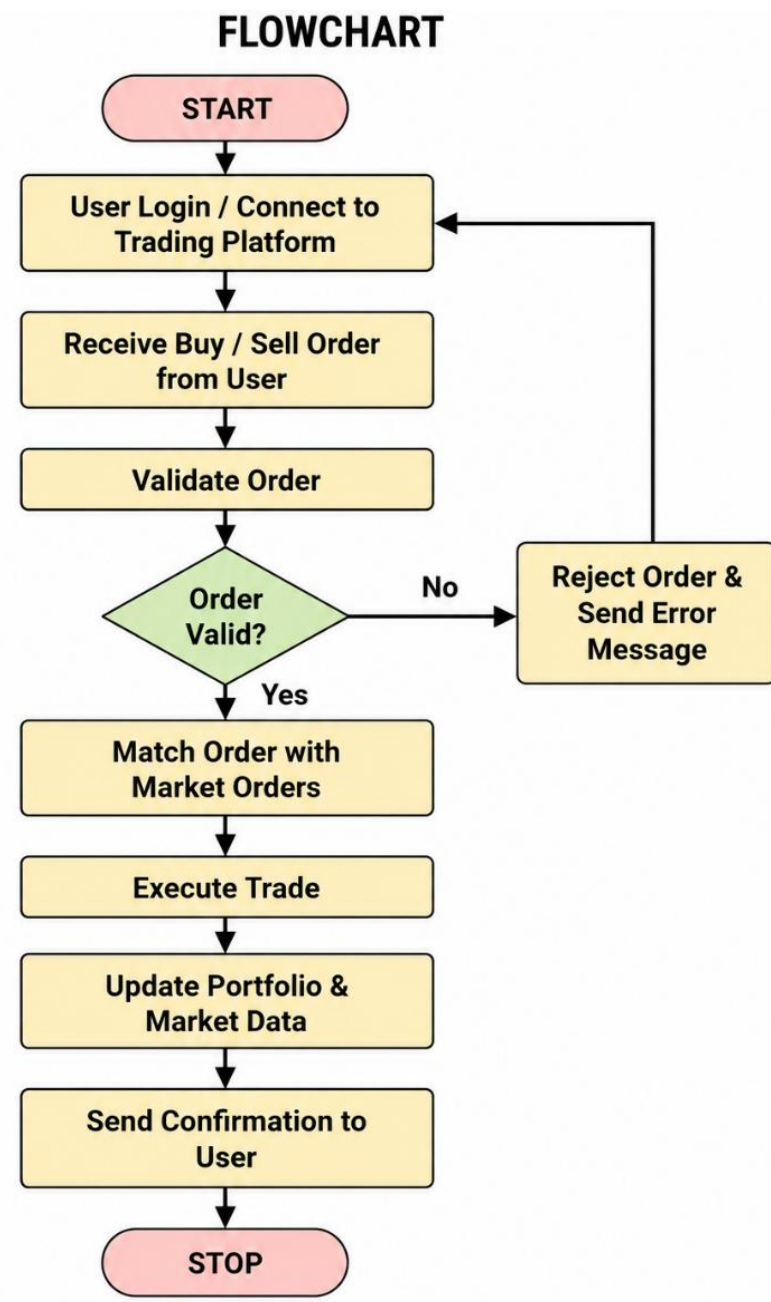
CPU Time

= $5,000,000 \times 1.2 \times 1 \text{ ns}$

= **6 ms**

Improvement

= **40% Faster**



Architectural Enhancements

Technique	Benefit
Cache Memory	Reduces memory access
Branch Prediction	Reduces branch delay
Instruction Pipeline	Executes multiple instructions
Out-of-Order Execution	Utilizes CPU efficiently
Prefetching	Fetches instructions early

Recommendation

- Increase cache size.
- Reduce branch instructions.
- Use superscalar processors.
- Enable pipelining.

Justification

Reducing CPI directly decreases execution time and improves trading performance during heavy workloads.

Question 3

Application Scenario

An industrial temperature controller uses an **8085 microprocessor**.

Occasionally, wrong temperature values are processed because incorrect addressing modes are used.

Identify

Different addressing modes affect how the processor accesses data.

Addressing Modes

Addressing Mode Description

Immediate	Data is part of instruction
Register	Data stored inside register
Direct	Address specified directly
Register Indirect	Address stored in register pair

Possible Causes

- Wrong memory location
- Wrong register selection
- Incorrect pointer values
- Sensor data overwritten
- Improper instruction sequence

8085 Sample Code

MVI A,25H

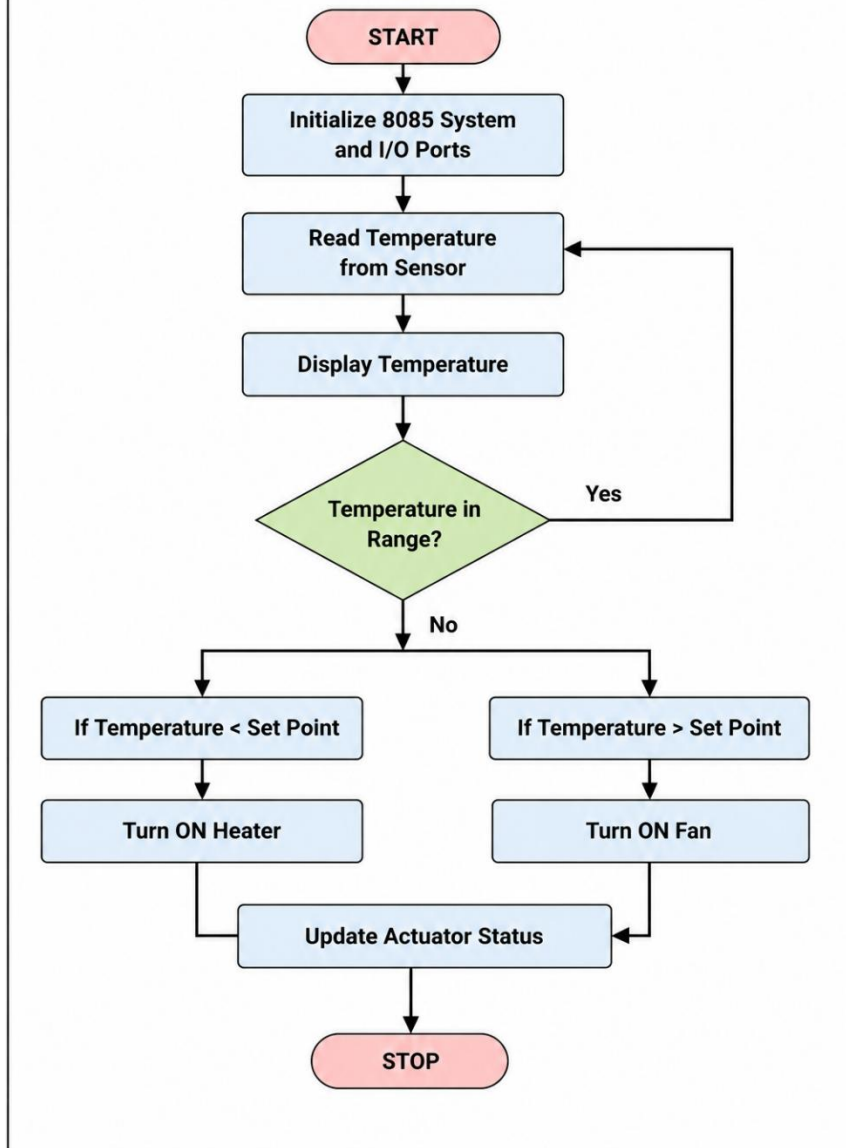
STA 2500H

LDA 2500H

OUT 01H

HLT

3. 8085 TEMPERATURE CONTROL SYSTEM FLOWCHART



Recommendation

- Use direct addressing for sensor memory.
- Validate memory addresses.
- Test every instruction carefully.

Justification

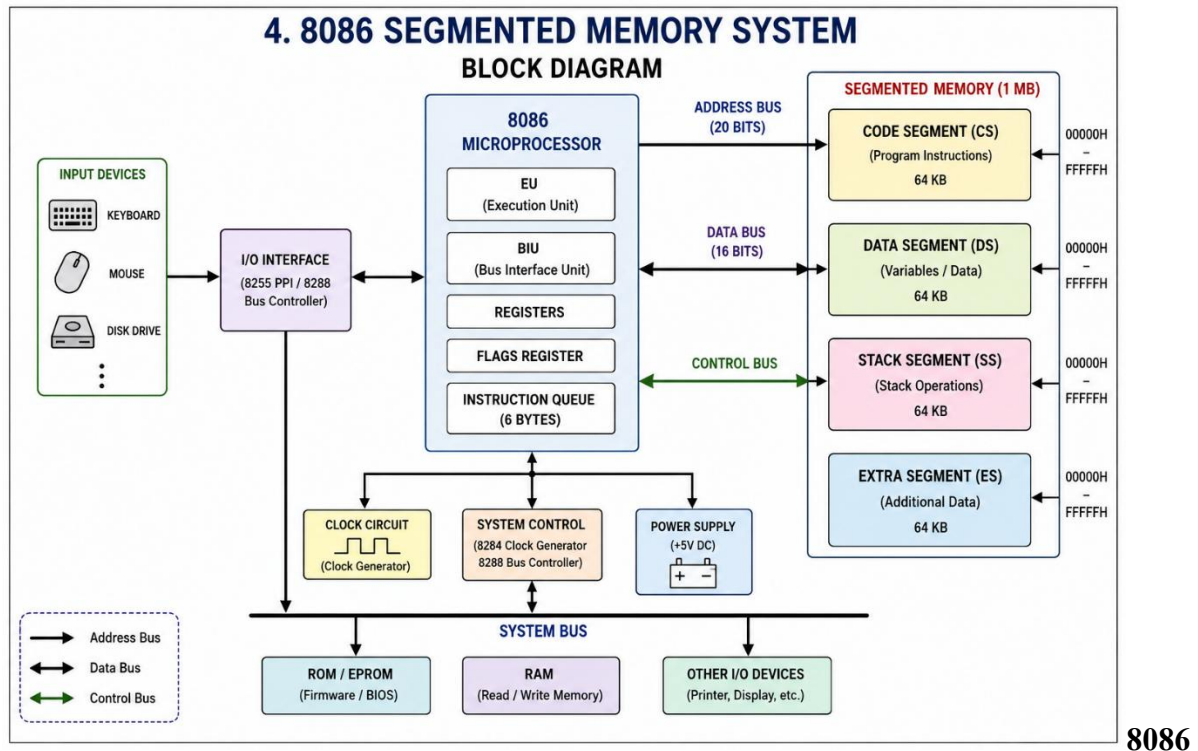
Using the correct addressing mode ensures accurate temperature measurement and reliable actuator operation.

Question 4

Application Scenario

An **8086 microprocessor** executes multiple programs using segmented memory.

Improper segment handling causes stack overflow and wrong data access.



Analyze

Each segment has a specific purpose.

- CS stores program instructions.
- DS stores variables.
- SS stores stack.
- ES stores extra data.

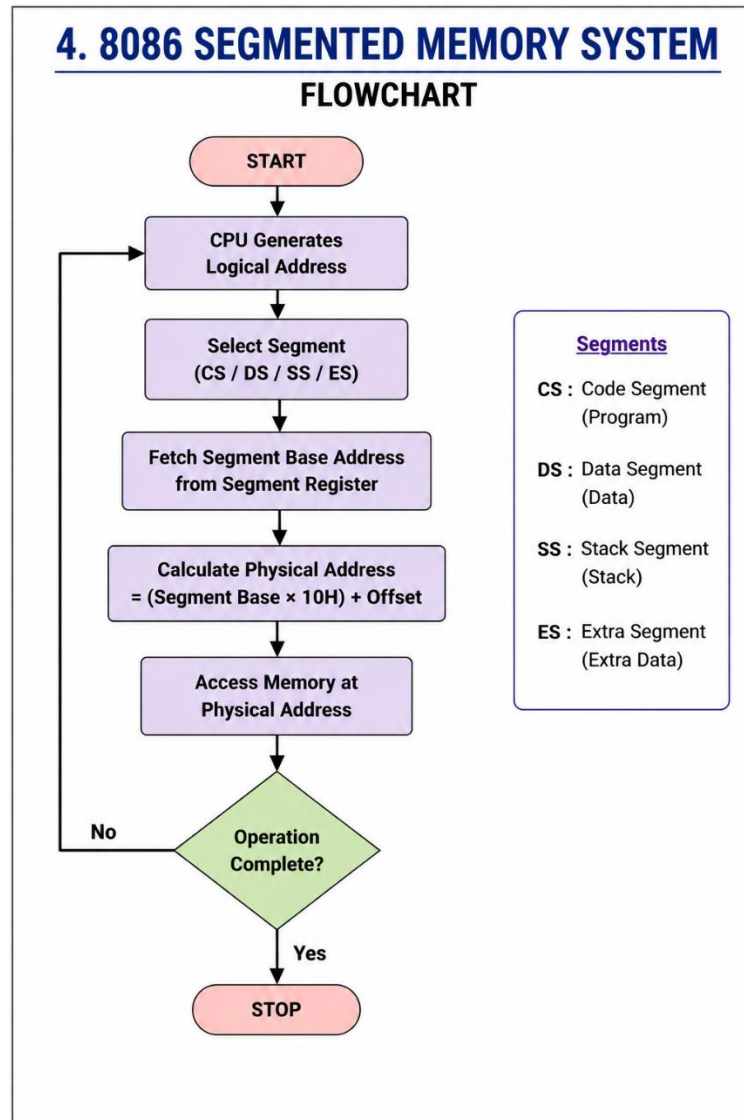
Incorrect segment register values cause the CPU to access wrong memory.

Problems

Problem	Reason
Wrong Data	Incorrect DS
Stack Overflow	Excess PUSH operations
Program Crash	Wrong SS:SP

Problem**Reason**

Memory Corruption Incorrect segment

**Recommendation**

- Initialize segment registers correctly.
- Monitor stack size.
- Use proper addressing modes.
- Avoid unnecessary PUSH instructions.

Justification

Correct segmentation ensures that every program accesses its own memory safely without corruption.

Question 5

Application Scenario

A communication system represents packet information in hexadecimal.

An engineer incorrectly converts hexadecimal values into binary, causing packet errors.

Importance of Number Conversion

Computers process data in binary, while engineers usually read hexadecimal.

Accurate conversion is essential for correct instruction execution.

Hexadecimal to Binary Table

Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111
3A	00111010

Example Conversion

Hexadecimal

3A(H)

Binary

0011 1010(B)

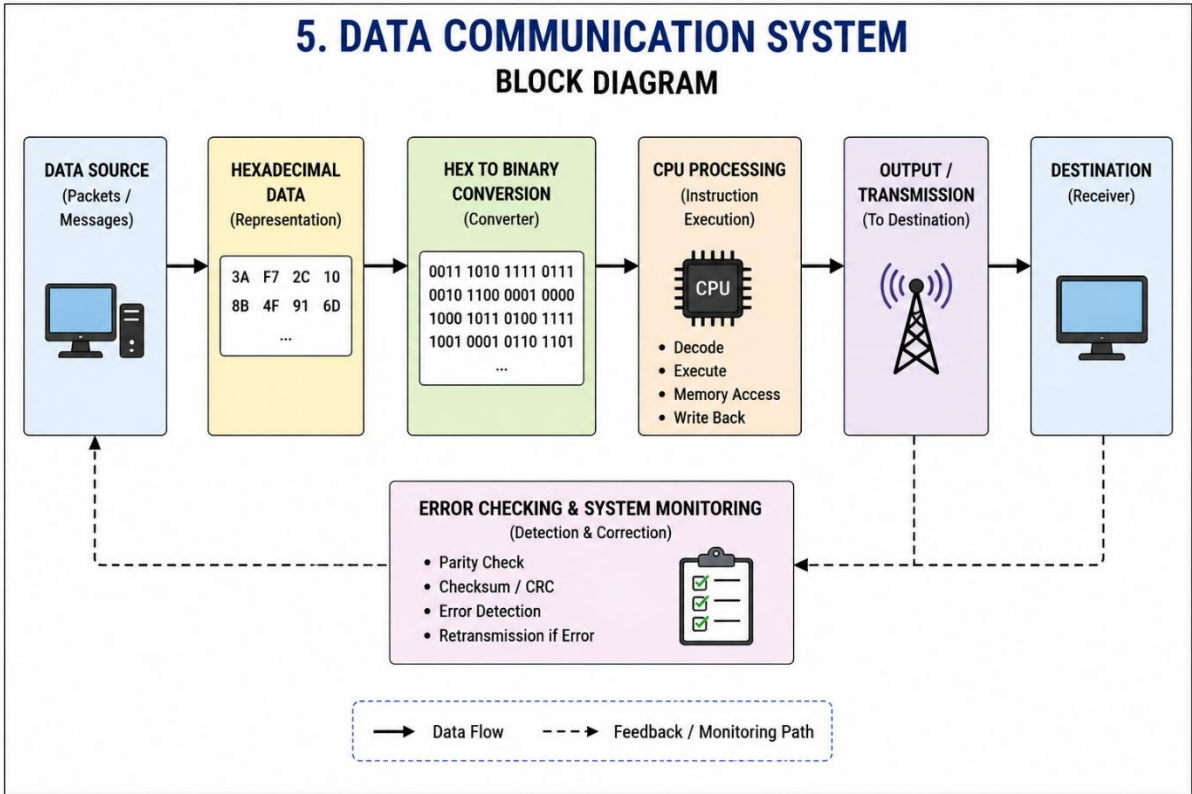
Effects of Wrong Conversion

- Wrong packet decoding
- Incorrect instruction execution
- Data corruption
- Communication failure

CPU Benchmarking

Benchmark	Purpose
Throughput	Number of packets processed

Benchmark	Purpose
CPI	CPU efficiency
Cache Miss Rate	Memory performance
Execution Time	Overall speed



Recommendation

- Use verified conversion libraries.
- Validate binary values.
- Perform CPU benchmarking regularly.
- Use parity/error checking.

Justification

Correct hexadecimal-to-binary conversion ensures accurate instruction execution and reliable communication. CPU benchmarking helps identify performance issues early and improves system reliability.